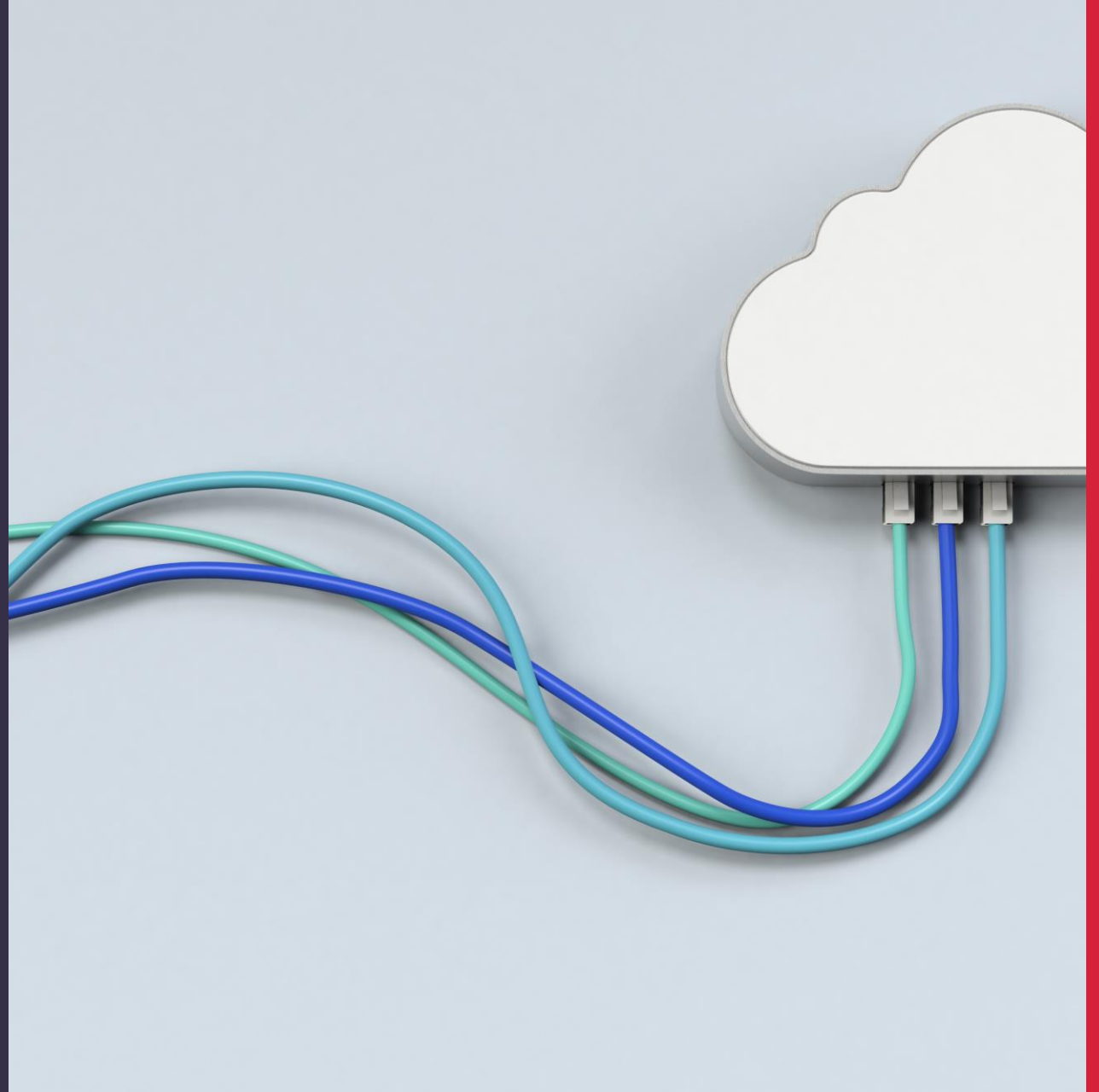


CloudOps Tools and Techniques Overview

WELCOME TO CLASS 1!



Goals of the day



Understand what we will achieve through this module.



Learn what Source Code Management is and why it is so important to CloudOps.



Learn most important commands and get hands on experience with git.

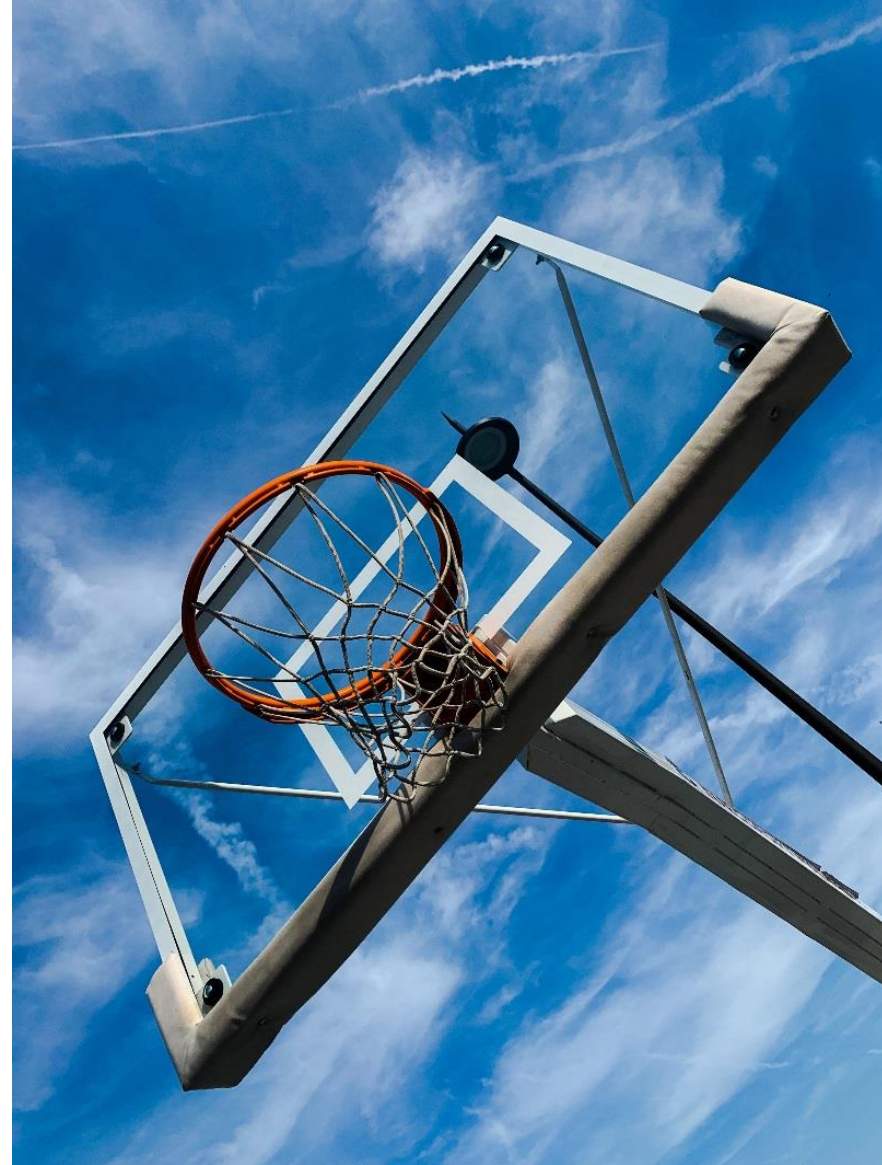


Photo by [David Becker](#) on [Unsplash](#)

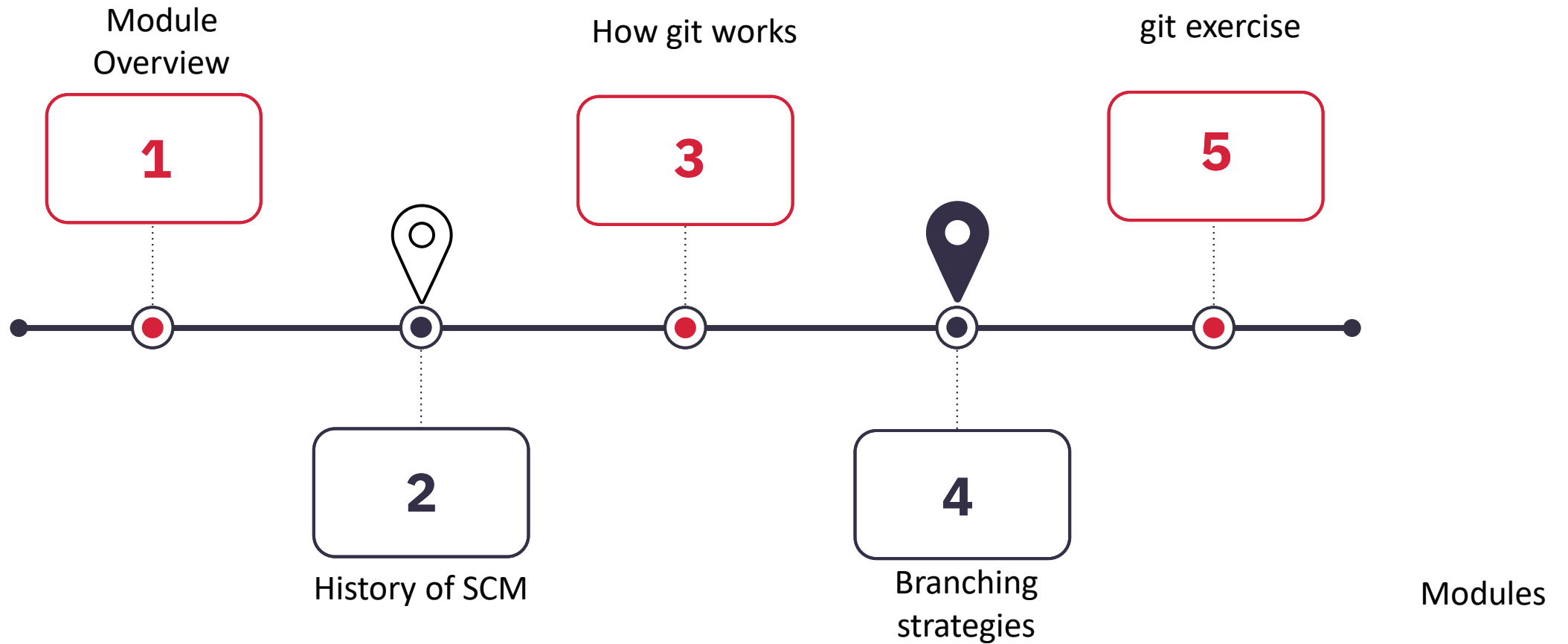
Agenda

- Instructor Introduction.
- Overview of the CloudOps Tools and Techniques Introduction module
- Brief history of source code management systems
- **Break**
- Using git to manage code effectively
- Understanding branching strategies and their implications.
- git exercise covering most common commands



Photo by David Becker on Unsplash

Journey Map

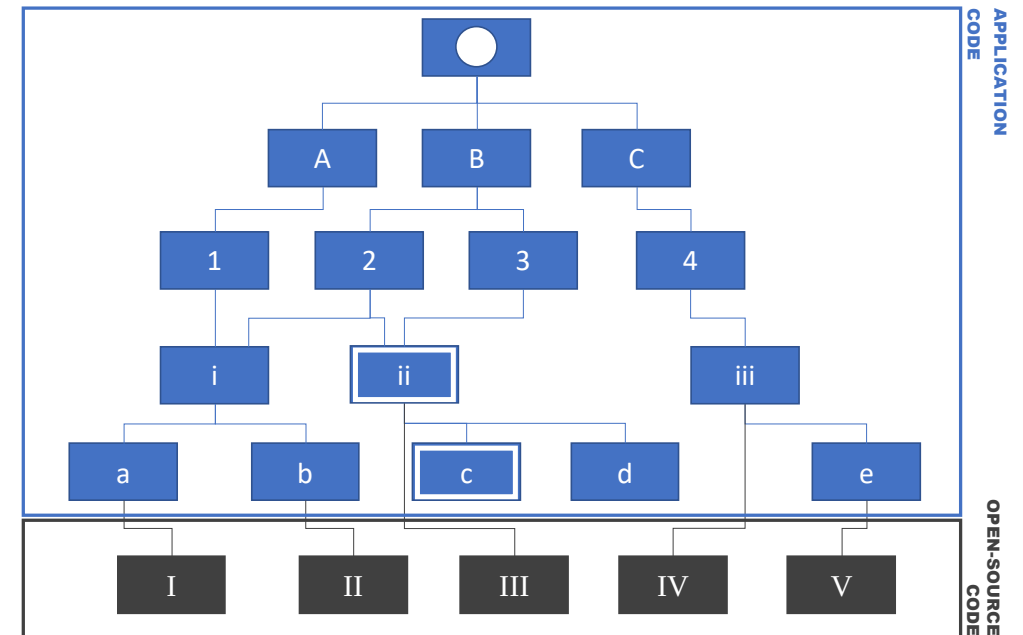


Overview

- What do we want learn in the module?
- Why is Infrastructure as Code (IaC) beneficial?
- How do we use IaC to create infrastructure?
- Beyond provisioning; there is configuration management.
- What is Continuous Integration and Continuous Delivery?
- Which tools support CI/CD and how?
- How to create great CI/CD pipelines?
- Vendor demo of popular CloudOps tooling.
- Metrics that measure effectiveness of DevOps/CloudOps.

Brief History of SCM tools (1 of 3)

- The structure of applications in code; comprised of several files containing code, which change over time as new features are added to the application.
- If one person is making changes to the code, then it is not too complicated.
- If many people are making changes to the code, they are in different locations and they are working on different requirements, with different delivery dates, then it gets complicated.



Brief History of SCM tools (2 of 3)

- Source Code Management tools bring order to the most complex of development teams.
- With so many concurrent changes, SCM helps to manage conflicts, providing a history of all changes for every file in the codebase. *
- Teams that survive keep copies of their critical code in SCM.
- Teams that thrive understand the power of SCM and store everything to install, run and test their application. **

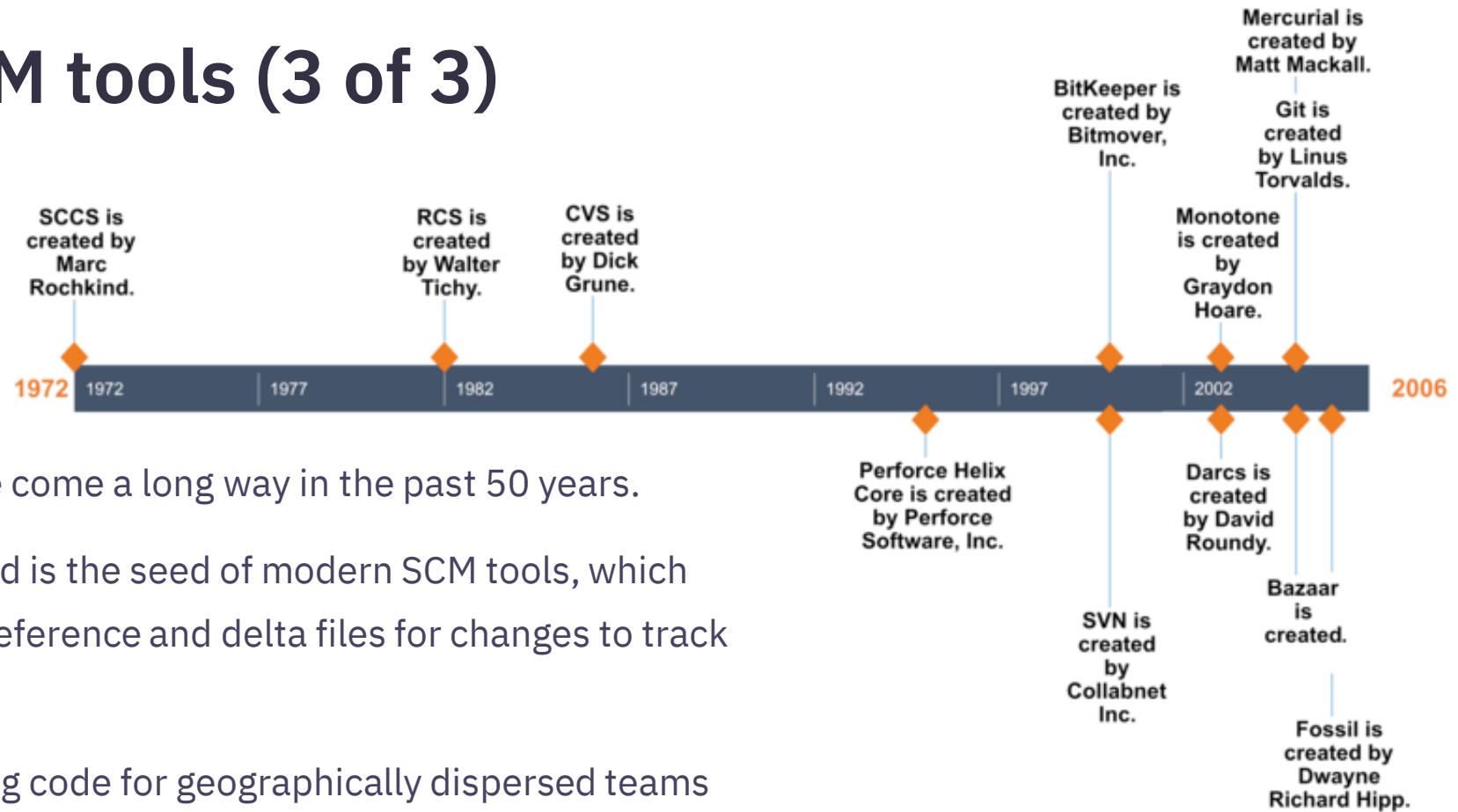


Photo by [Mel Poole](#) on [Unsplash](#)

* <https://www.atlassian.com/git/tutorials/what-is-version-control>

** Continuous Delivery; Humble, Jez & Farley, David; Addison Wesley,; 2011

Brief History of SCM tools (3 of 3)



- Source Code Management tools have come a long way in the past 50 years.
- SCCS was written in C at Bell Labs and is the seed of modern SCM tools, which created a copy of the source file for reference and delta files for changes to track the state of each file.
- CVS was the 2nd generation, managing code for geographically dispersed teams
- SVN created a local copy of the codebase, with a central repository so that you needed a network connection to commit code.
- Git, written by the creator of Linux, in response to the shortcomings of all other available options. *

<https://initialcommit.com/blog/Technical-Guide-VCS-Internals>

* <https://www.linuxfoundation.org/blog/blog/10-years-of-git-an-interview-with-git-creator-linus-torvalds>

Essential practices in SCM use #1 *

- Don't check in on a broken build
- Always run all commit tests locally before committing
- Wait for commit tests to pass before moving on
- Never go home on a broken build
- Always be prepared to revert to a previous version
- Time box fixing before reverting
- Don't comment out failing tests
- Take responsibility for all breakages the result from your changes
- Test Driven Development



Photo by [Annie Spratt](#) on [Unsplash](#)

* Continuous Delivery; Humble, Jez & Farley, David; Addison Wesley; 2011

Essential practices in SCM use #2 *

- Commit often
- Ensure you're working from the latest build
- Make detailed notes
- Review changes before committing
- Use branches
- Agree on a Workflow



Photo by [Annie Spratt](#) on [Unsplash](#)

* <https://www.atlassian.com/git/tutorials/source-code-management>

Essential practices in SCM - which do you prefer?

Atlassian

- Commit often
- Ensure you're working from the latest build
- Make detailed notes
- Review changes before committing
- Use branches
- Agree on a Workflow



Photo by [Annie Spratt](#) on [Unsplash](#)

Continuous Delivery book

- Don't check in on a broken build
- Always run all commit tests locally before committing
- Wait for commit tests to pass before moving on
- Never go home on a broken build
- Always be prepared to revert to a previous version
- Time box fixing before reverting
- Don't comment out failing tests
- Take responsibility for all breakages the result from your changes
- Test Driven Development

Break (15 Minutes)



How to install git

- There are many different git-based version control platforms available, including:
 - GitHub
 - GitLab
 - Azure DevOps/Bitbucket
- Of the public offerings, GitHub has the larger market share. For this reason, we will use GitHub as part of this course.
- Microsoft acquired GitHub in 2018. *

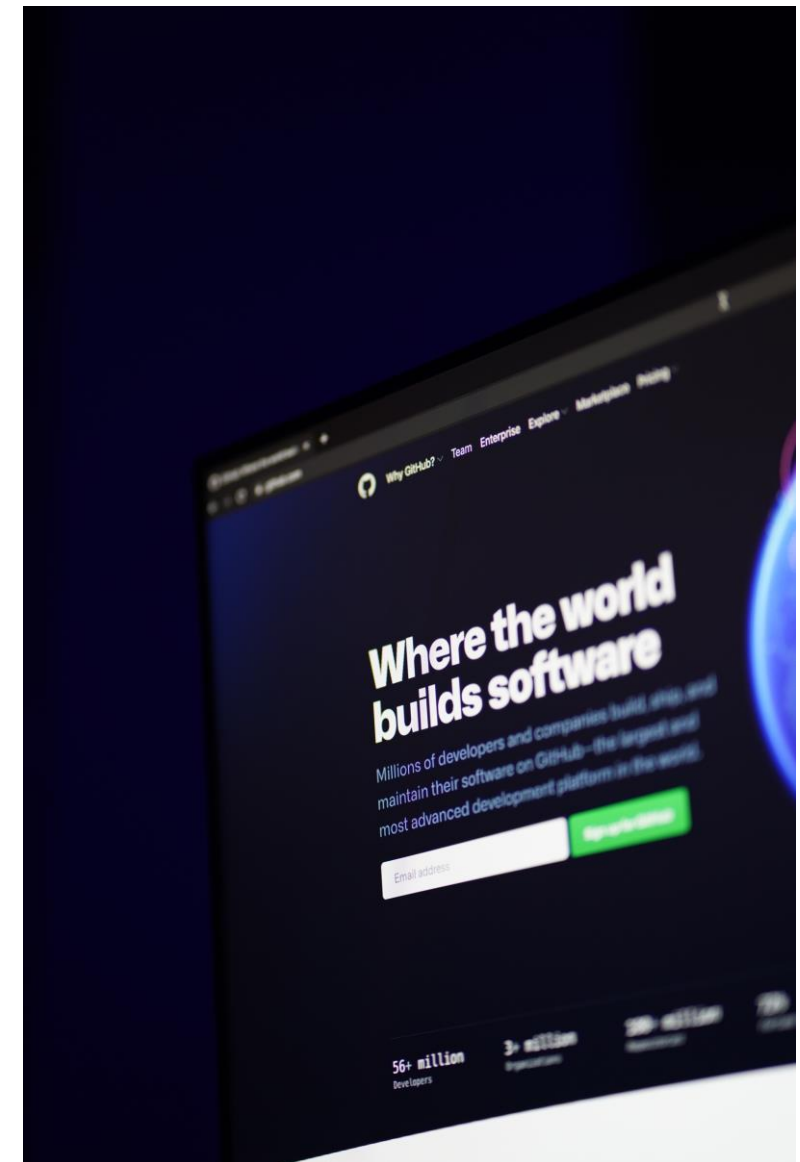


Photo by [Mohammad Rahmani](#) on [Unsplash](#)

* <https://news.microsoft.com/announcement/microsoft-acquires-github/>

How to install git (con't)

- There are several options for installing git on your laptop.
 - GUI driven
 - GitHub Desktop
 - IDE integration
 - Visual Studio Code [git comes preinstalled]
 - Command line
 - Git Bash [for Windows OS]
 - Git from shell or terminal [Linux or MacOS]
- Which one you chose should consider how comfortable you are with command line driven interfaces, and how likely you are to use another version control platform e.g, GitLab.
- Visual Studio Code is a good option.

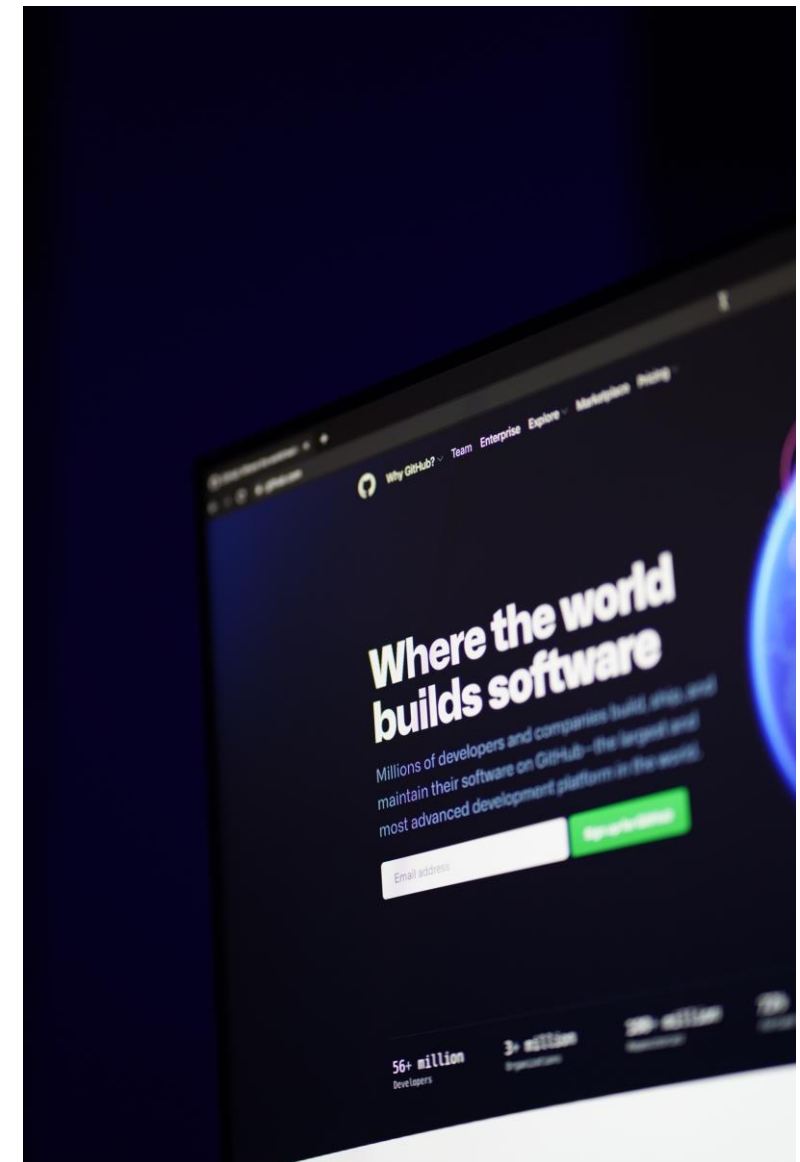


Photo by [Mohammad Rahmani](#) on [Unsplash](#)

How git works



This Photo by Unknown Author is
licensed under CC BY-SA-NC

How git works – commits (1 of 3)

Commits are the building blocks of git. When you commit, you are telling git that you have completed a logical unit of change and that you want to preserve it.

Git creates a snapshot of your code when you make a commit.

Each commit has a unique identifier, which is a SHA1 code generated based on the files in that commit.



This Photo by Unknown Author is licensed under CC BY-SA

How git works – commits (2 of 3)

Commit messages are mandatory.

As you continue to make changes and commit regularly, you build up a commit history.

Best practice is to commit often with meaningful commit messages, which are visible in the commit history.

If you find a problem that you cannot unravel, you can easily revert to a specific commit and resume work from that point in the timeline.



This Photo by Unknown Author is licensed under CC BY-SA

How git works – commits (3 of 3)

Making sequential commits to build a history that can be navigated is quite straightforward, if there's only one person working on the code.

Where all changes are committed to one timeline, it is called the mainline or master copy of the code.

In reality, whole teams of people need to implement a wide range of changes on a codebase at the same time.

This is where branching comes in.



This Photo by Unknown Author is licensed under CC BY-SA

How git works - branches

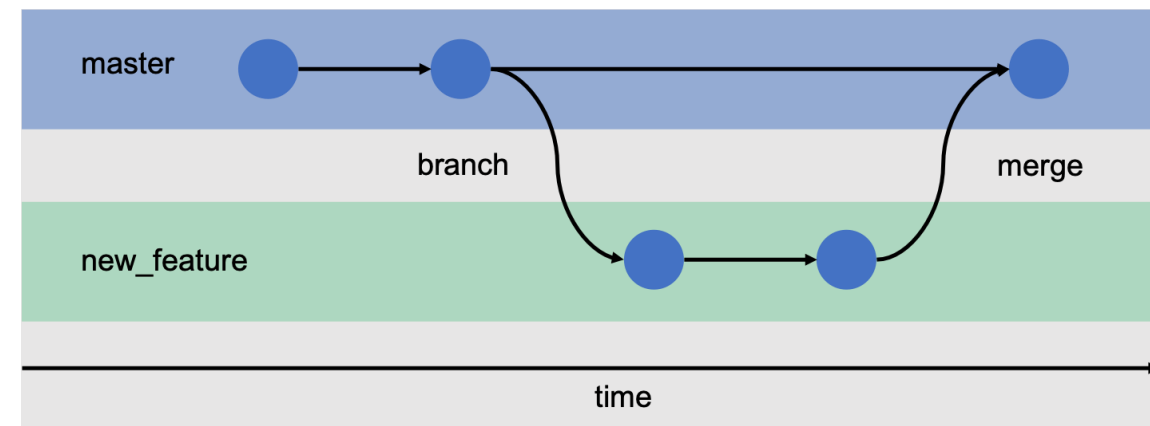
Branching means diverging from the mainline so that work can continue separately and without having any impact on the mainline.

The mainline is typically called master branch

This approach allows experimentation in the codebase or development of new features independent of the master.

Once work on a branch is complete it can be merged back to master.

This is an immensely powerful feature, which forms the foundations of great skyscrapers.



This Photo by Unknown Author is licensed under CC BY-SA-NC

How git works – remote vs local

We've covered commits, branches and merges.

All of these activities can take place on a local git instance, but to collaborate we need to use a remote instance that is accessible by others.

We use GitHub, GitLab, Azure Devops or Bitbucket for hosting remote instances of our code as public or private git repos.

Authentication, authorization, tokens, https and ssh access are all managed by these platforms.



[This Photo](#) by Unknown Author is licensed under [CC BY](#)

How git works – command cheat sheets

git's command line interface (CLI) is very effective.

Here are git commands for basic local workflow on the topics we have covered.

Command	Purpose
git init	Creates a new local repository
git add <file name>	Add a new file to git for commit
git rm <file name>	Remove a file to exclude from commit
git commit -a	Commit a file locally, add comments
git branch	List local repo branches
git checkout <branch name>	Switch to the specified branch name
git checkout -b <branch name>	Create and switch to specified branch
git log	View commit history
git merge <branch name>	Merge the named into current

How git works – command cheat sheets (con't)

Here are git commands for basic remote and advance local workflow on the topics we have covered.

It takes time to become skilled in using git CLI.

Command	Purpose
git clone <repo_url>	Copies existing git repo
git pull	Download a remote repo, updating local repo.
git push	Push local commits back to origin or master
git status	Status of your local vs origin or remote
git diff	For current branch, show uncommitted changes
git diff <name>	Compare current branch with a specified branch
git branch -d <name>	Delete local specified branch
git push origin --delete <name>	Delete remote specified branch

Branching Strategies



This Photo by Unknown Author is licensed under CC BY-NC

Branching Strategies (1 of 2)

If you are familiar with software development delivery, you have likely heard of branching.

It is a topic that generates a lot of debate in projects and teams because it is so impactful to the Software Development Lifecycle (SDLC) workflow.

For teams of more than 2-3, it is critical that a specific strategy has been adopted and that it clearly understood by developers and release management.

Now that we have learned the fundamentals of git, let's turn our attention to branching strategy.



This Photo by Unknown Author is licensed under CC BY-NC

Branching Strategies (2 of 2)

One of git's strengths is the flexibility it offers.

This flexibility means there is no prescribed workflow for use of git.

In choosing a workflow, the following criteria should be considered:

- Does the workflow scale as the team grows?
- Is it easy to undo changes?
- Can code changes be reviewed by others before go live?
- Is the workflow complex to understand?



Branching Strategies – Centralized Workflow

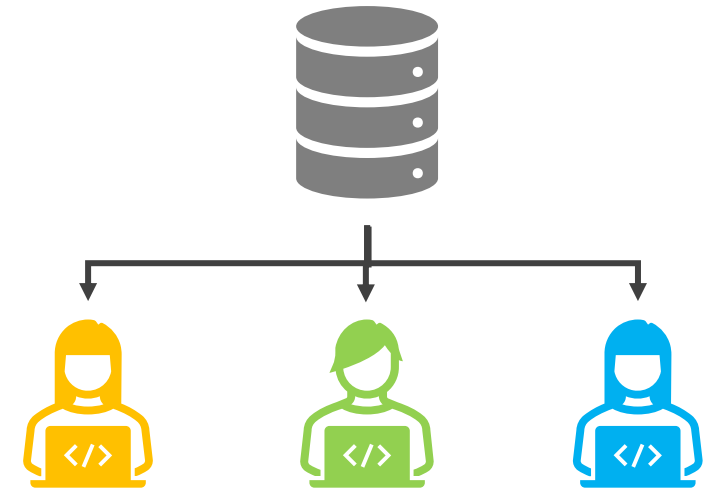
This workflow uses a centralized repository as the entry point for all changes.

This workflow uses one branch only – master.

Developers clone the remote repo to local, then work on their changes, committing regularly to local.

Once local work is complete, they can push changes back to remote.

No prescribed pull request or forking approach defined.



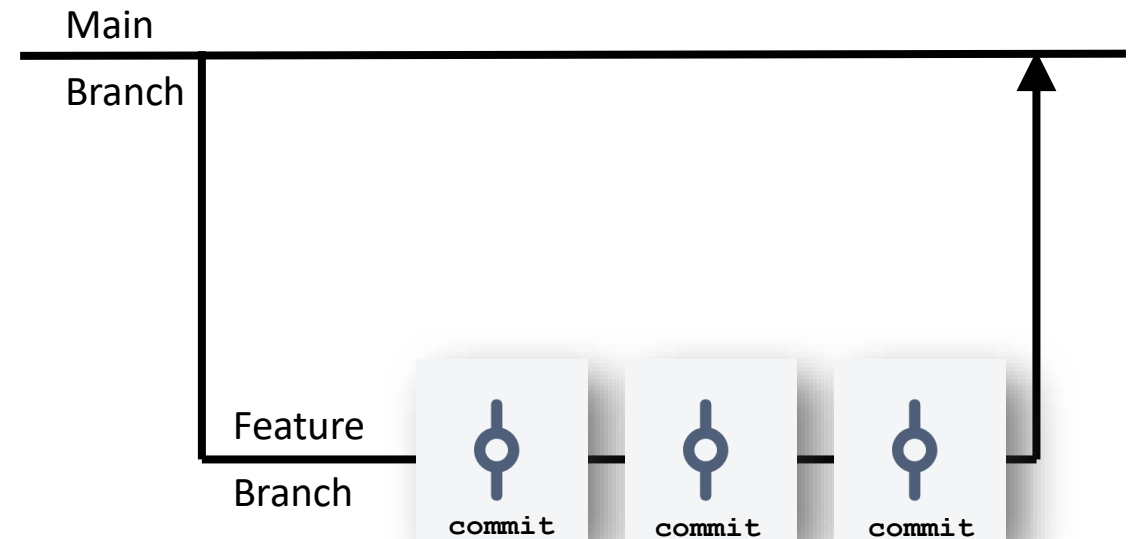
Branching Strategies – Feature Branch Workflow (1 of 3)

The workflow is based on distinct branches for each feature.

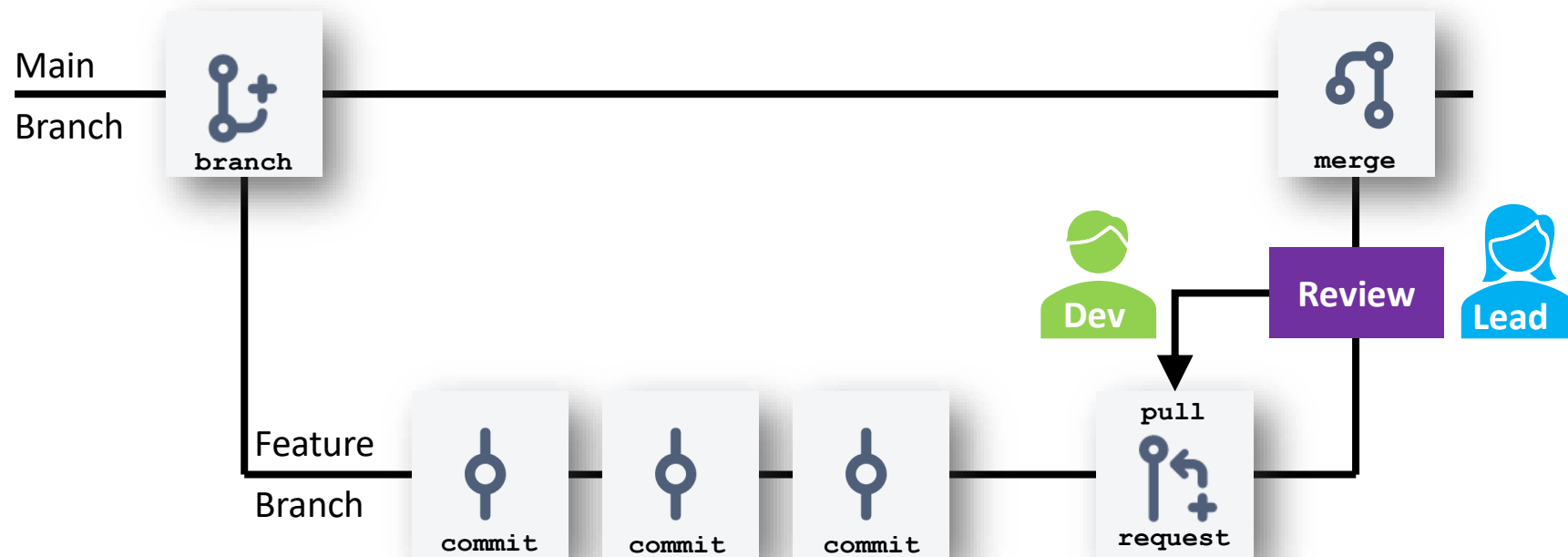
When the developer starts a new task, they create a feature branch from master or develop.

All changes are completed locally on the feature branch. Once complete, the feature branch is pushed back to remote.

At this point, the feature branch can be merged into the main branch for deployment.



Branching Strategies – Feature Branch Workflow (2 of 3)



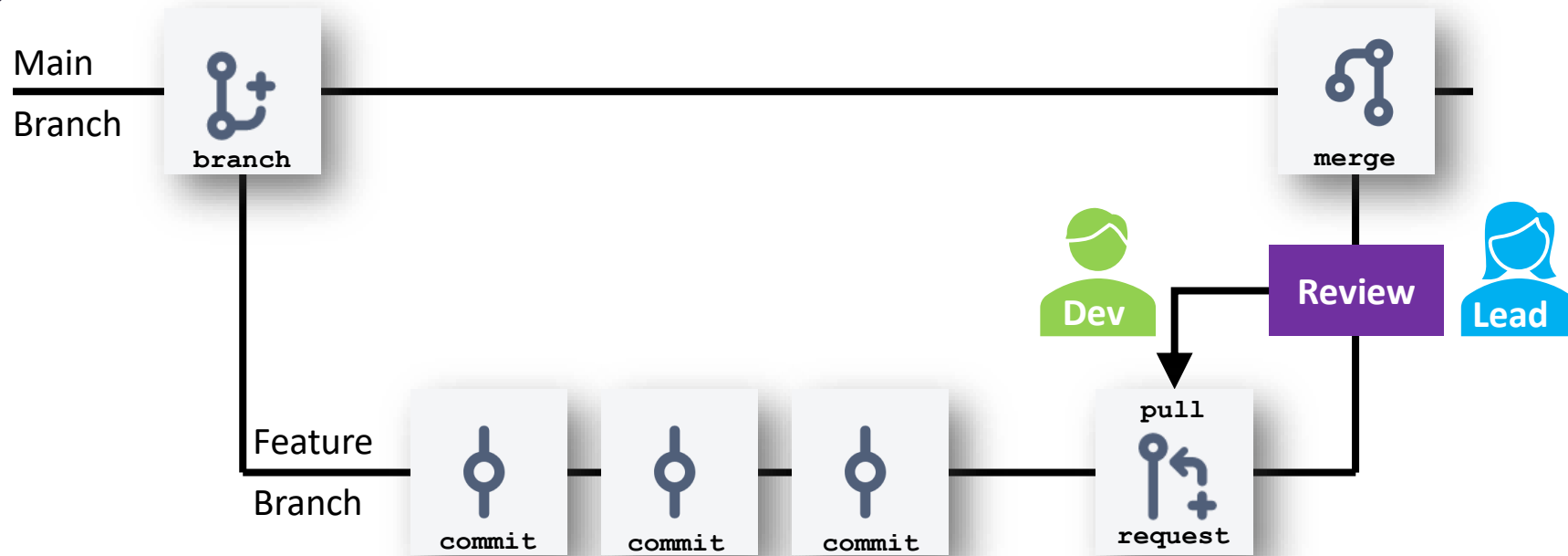
Pull requests

It is common to use pull requests to control the merge of code from feature branch to main branch.

Developer raises a pull request, requesting that their feature branch is merged into main branch.

The pull request is reviewed typically review by senior team member as part of a formal code review.

Branching Strategies – Feature Branch Workflow (3 of 3)



Pull requests

The request can be denied based on design, code quality, etc., and returned to requester with comments. This back and forth is documented in the request.

Security, Architecture and QA can also be included as reviewers and can block merges.

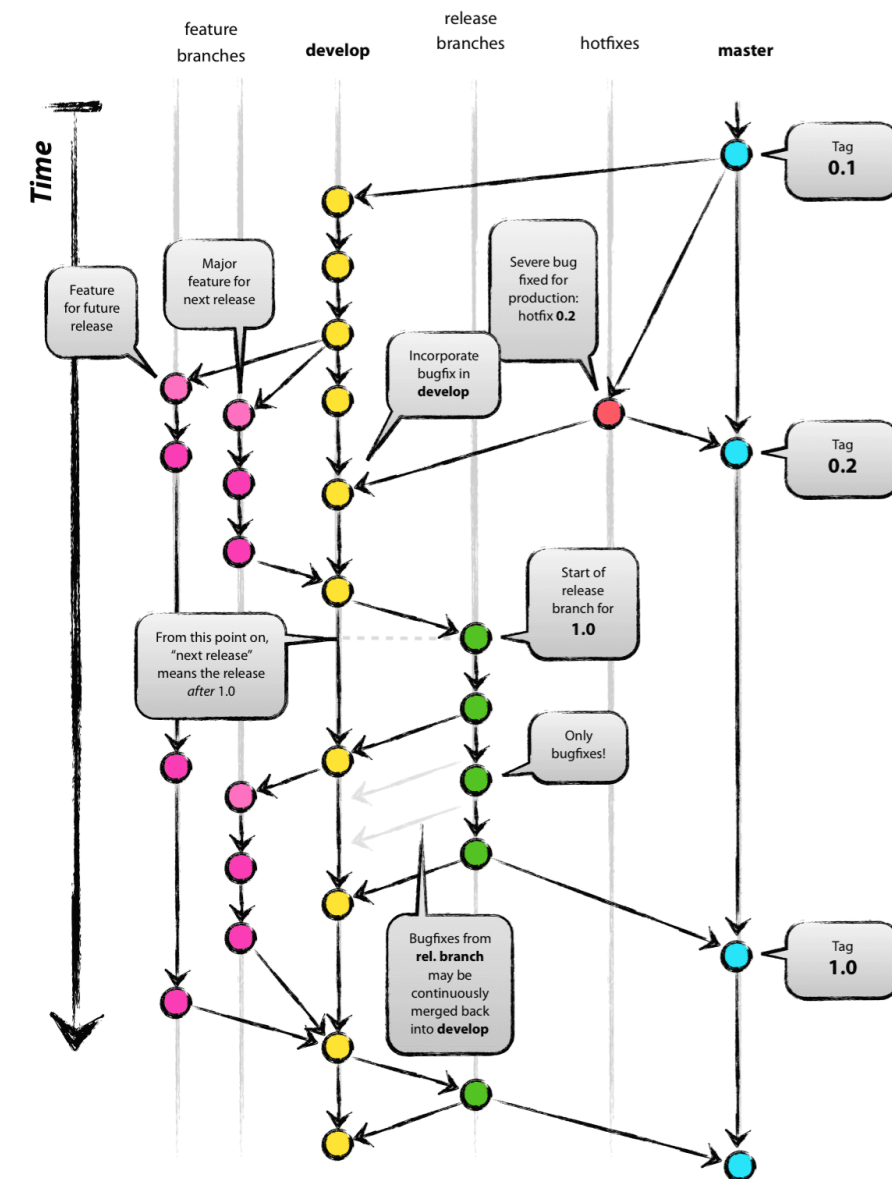
Branches often mean to control progression into higher environments, with a clear audit trail built in.

Branching Strategies – Gitflow

Although seemingly quite complex, Gitflow is a very popular branching strategy for teams with long running project releases or a fixed delivery cadence.

This pattern has a very well-defined branching hierarchy, which outlines how branches are created/merged and clearly illustrates how to progress a commit to production (master) branch.

Develop is created from master initially and is used as an integration branch between feature and release branches.



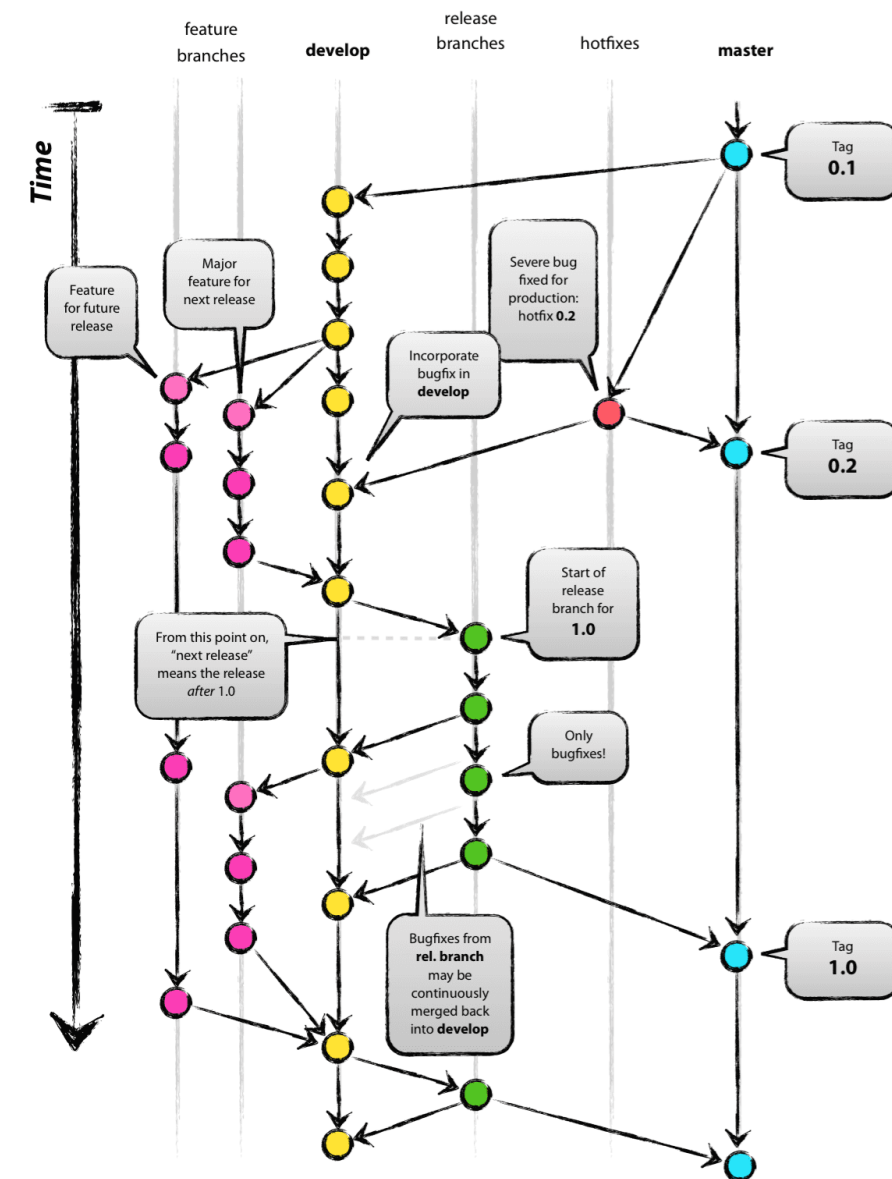
<https://nvie.com/posts/a-successful-git-branching-model/>

Branching Strategies – Gitflow (con't)

Once a release branch is created, or cut as sometimes described, it should only be updated with bugfixes, which in turn are applied back to develop.

When ready for Go Live, the release branch is merged to both develop and master

On release, the last commit should be tagged, and the release branch should then be deleted.



<https://nvie.com/posts/a-successful-git-branching-model/>

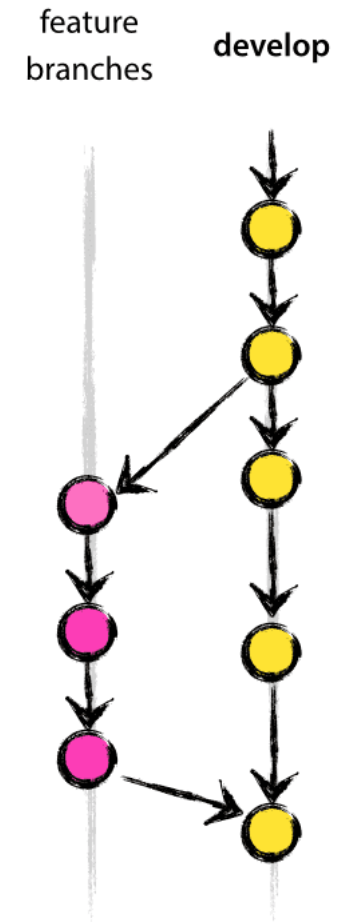
Branching Strategies – Summary

The Centralized workflow is quite restrictive and is generally considered a good fit as a first step for teams migrating from an older SCM and learning git.

Feature branches are widely used and when short lived, can adequately support Continuous Integration.

Gitflow builds on feature branches and is effective for larger teams working to fixed dates, where Continuous Integration is not the focus.

GitHub is another extension of feature branches, where GitHub is the SCM used, and the deployment model is Continuous Delivery.



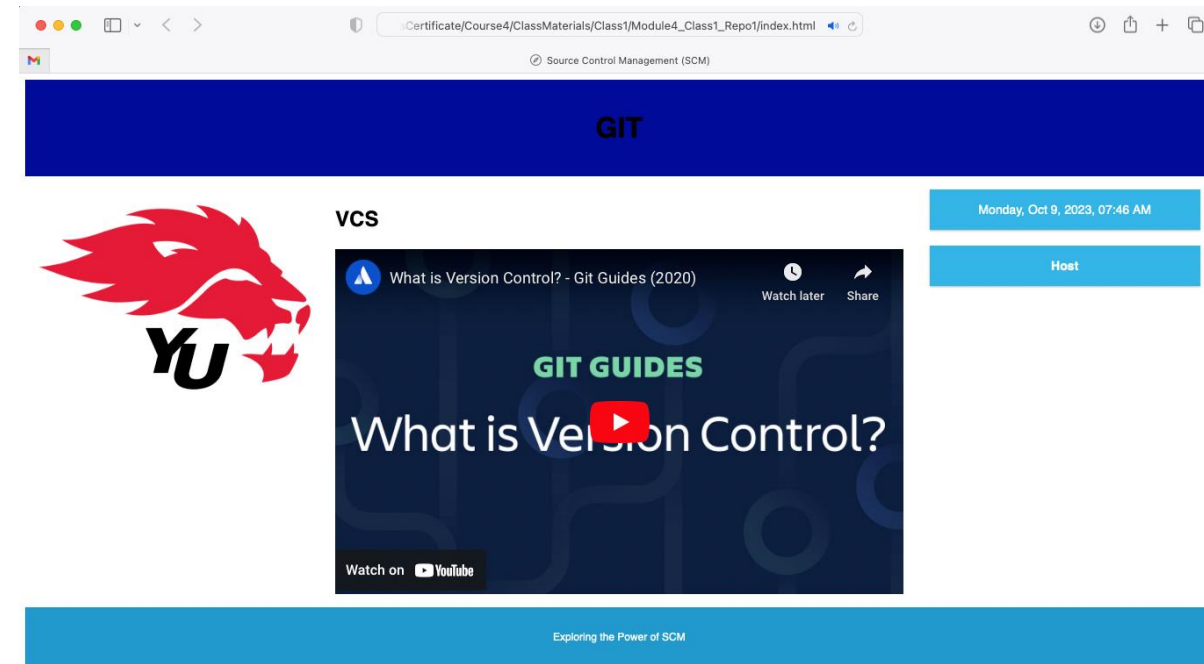
<https://nvie.com/posts/a-successful-git-branching-model/>

Exercise 1 (1 of 3)



In this exercise, you clone a repo, create your own feature branch and merge that branch back to main

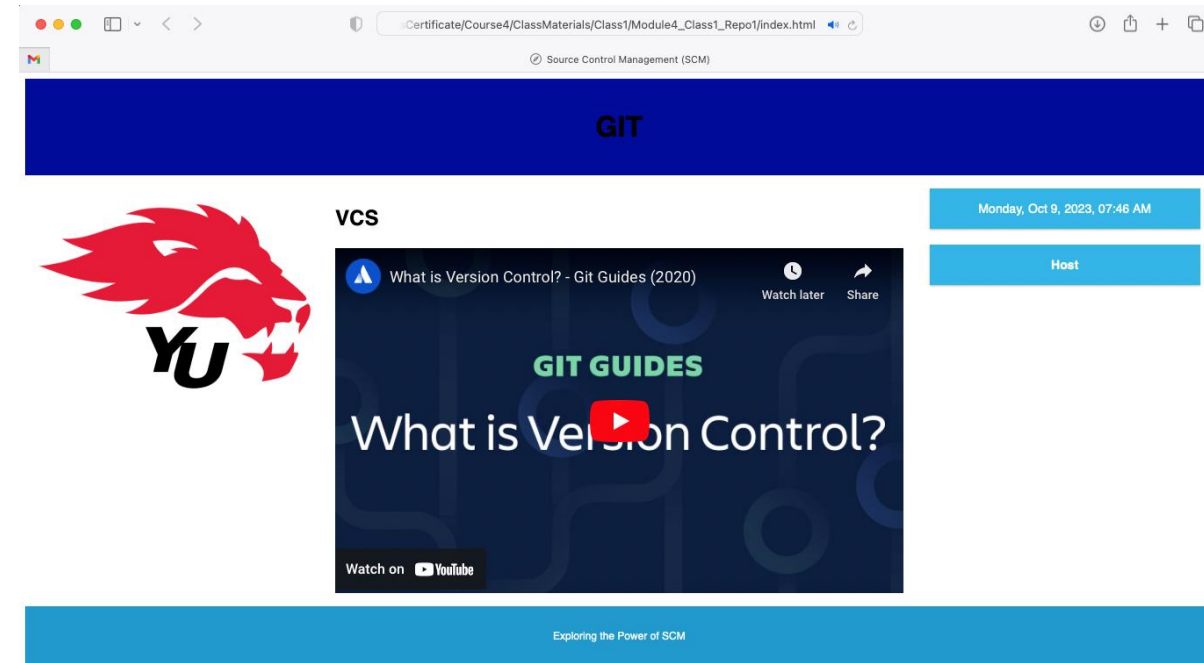
1. Fork the repo to your GitHub account.
2. Clone from your remote to your local.
3. Verify that you can view the index.html in your browser.
4. Create a feature branch from master.
5. Switch to that branch. Modify the index.html.



Exercise 1 (2 of 3)



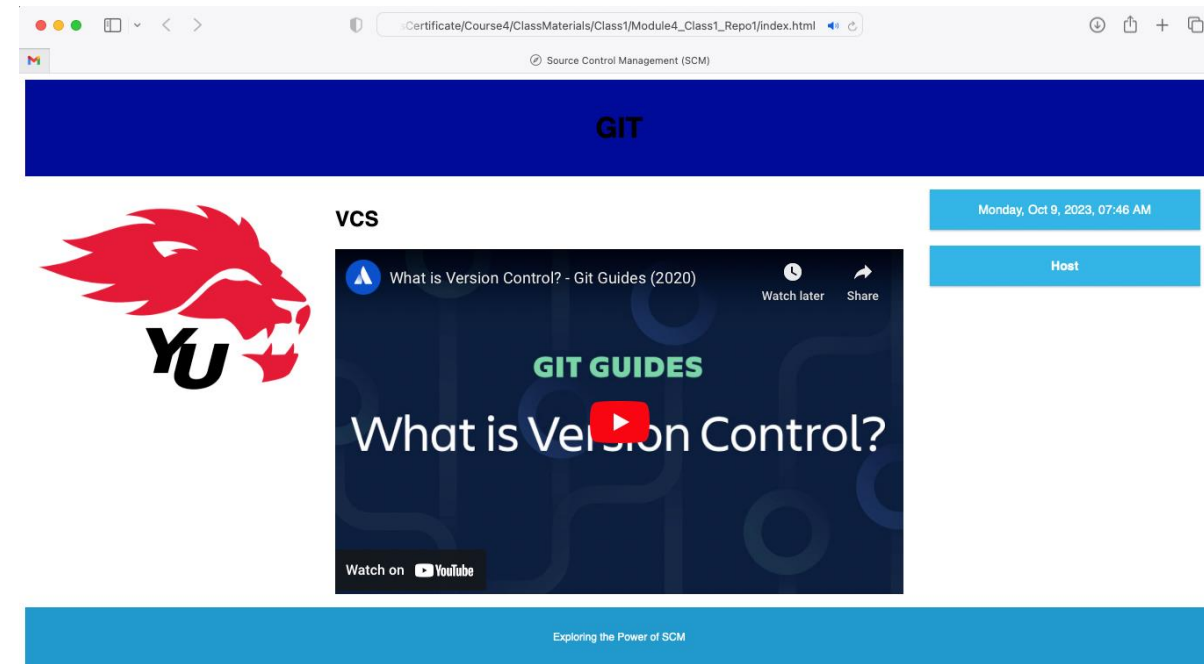
5. Switch to that branch. Modify the index.html.
6. Verify that the changes load in the browser.
7. Commit the code to your branch.
8. Push your code to your GitHub account.
9. Browse to your feature branch in your GitHub account. Verify that your changes to index.html are visible.
10. In your local, merge your feature branch into master.



Exercise 1 (3 of 3)



11. Verify that the index.html changes are visible in the master branch.
12. Push changes to remote.
13. Browse to your master branch in your GitHub account. Verify that your changes to index.html are visible.
14. In your local, delete the feature branch.
15. Push the delete to remote.
16. Verify that the feature branch is delete in your GitHub account.



Wrap up

- Q & A
- What's 1 thing you learned?
- What's 1 challenge?



References

- Images [Slide 1] used under license from Microsoft PowerPoint stock images.
- Image [Slide 12] 2020 "Woman Sitting on Bench Looking at Mountains" by Yan Krukau used under free license from Pexels.